

Д35: Реализация межсервисного взаимодействия посредством очередей сообщений

Студент: Зеленков Евгений

Группа: БР2.2

Тема: подключение RabbitMQ к микросервисному backend-приложению бронирования ресторанов

Цель работы

Цель работы - подключить брокер сообщений к микросервисной версии приложения и реализовать асинхронное межсервисное взаимодействие через очередь сообщений.

Выбранный брокер

Для реализации выбран RabbitMQ. Он подходит для текущей архитектуры, потому что приложение уже использует доменные события и outbox-заготовку из ЛР2. RabbitMQ добавлен в docker-compose.yml как инфраструктурный сервис рядом с PostgreSQL.

Параметры локального запуска:

Компонент	
Значение	
AMQP URL	amqp://booking_user:booking_password@localhost:5672
Management UI	http://localhost:15672
Exchange	booking.events
Exchange type	topic

Реализованные события

В приложении используются доменные события единого формата:

```
{  
  "eventId": "uuid",  
  "eventVersion": 1,  
}
```

```
"eventType":  
"review.verified",  
"occurredAt": "2026-05-  
20T10:00:00.000Z",  
"payload": {}  
}
```

Реализованы события:

Событие

Publisher

Consumer

review.verified

review-service

restaurant-catalog-service

reservation.status_changed

reservation-service

пока публикуется для последующих подписчиков

Сценарий review.verified

Администратор подтверждает отзыв через PATCH /api/v1/reviews/{reviewId}/verification.

review-service сохраняет изменение в своей БД.

review-service сохраняет событие в таблицу outbox_events.

review-service публикует событие review.verified в RabbitMQ exchange booking.events.

restaurant-catalog-service получает сообщение из очереди restaurant-catalog.review-verified.v2.

Catalog-service проверяет идемпотентность через таблицу processed_messages.

Catalog-service обновляет агрегат рейтинга в restaurant_rating_aggregates.

Catalog-service пересчитывает поле restaurants.rating.

Так review-service не вызывает restaurant-catalog-service синхронно при модерации.

Сбой catalog-service не отменяет успешную модерацию отзыва, а сообщение остается в очереди RabbitMQ.

Надежность обработки

Для надежности добавлены следующие механизмы:

сообщения публикуются как persistent;

publisher использует RabbitMQ ConfirmChannel и ждет подтверждения брокера через waitForConfirms;

exchange и queue объявлены как durable;

consumer использует ack только после успешной обработки;

для ошибок обработки настроена dead-letter queue restaurant-catalog.review-verified.v2.dlq, поэтому некорректные сообщения не попадают в бесконечный цикл повторной обработки;

таблица processed_messages защищает от повторной обработки одного eventId;

поле business_key в processed_messages защищает от повторной обработки одного отзыва при разных eventId;

таблица `restaurant_rating_aggregates` хранит сумму рейтингов и количество подтвержденных отзывов.

Таблица `outbox_events` используется как основа для transactional outbox: сервисы сохраняют событие перед публикацией и отмечают `publishedAt` после успешной отправки в RabbitMQ. Отдельный outbox worker с retry-политикой в текущей версии не выделен.

Изменения в коде

Основные изменения внесены в `labs/lab2/app`:

добавлена зависимость `amqp-lib`;

добавлены env-переменные `RABBITMQ_URL` и `RABBITMQ_EXCHANGE`;

в `docker-compose.yml` добавлен сервис `rabbitmq`;

`ConsoleEventPublisher` заменен на `RabbitMqEventPublisher`;

`review-service` публикует событие `review.verified`;

`reservation-service` публикует событие `reservation.status_changed`;

`restaurant-catalog-service` запускает RabbitMQ consumer при старте;

в `catalog-service` добавлены таблицы `processed_messages` и `restaurant_rating_aggregates`.

Запуск

```
cd labs/lab2/app
```

```
cp .env.example .env
```

```
docker compose up -d postgres rabbitmq
```

```
npm install
```

```
DATABASE_SYNCHRONIZE=true npm run seed
```

```
npm run build
```

Сервисы запускаются отдельными процессами:

```
npm run dev:identity
```

```
npm run dev:catalog
```

```
npm run dev:menu
```

```
npm run dev:reservation
```

```
npm run dev:review
```

```
npm run dev:gateway
```

Вывод

В ходе работы к микросервисному backend-приложению был подключен RabbitMQ.

Межсервисное взаимодействие для обновления рейтинга ресторана после модерации отзыва переведено на асинхронную обработку через очередь сообщений. Также сохранена outbox-заготовка и добавлена идемпотентная обработка событий на стороне consumer.